



ПРОФЕССИОНАЛЬНЫЙ
ТЕХНИЧЕСКИЙ
АНАЛИЗ В METATRADER 5



УСТАНОВИ СЕЙЧАС ↓

MetaTrader 5 / Примеры

Свинговые экстремумы и откаты в MQL5 (Часть 2): Автоматизация стратегии с помощью советника

2

Hlomohang John Borotho

118

Содержание

1. [Введение](#)
2. [Обзор индикатора](#)
3. [Начало работы](#)
4. [Демонстрация индикатора](#)
5. [Заключение](#)



Введение

Эта статья предназначена для тех, кто пытается выявлять развороты на ранней стадии, входит против импульса или не до конца понимает расположение истинного экстремума. Цена редко разворачивается на пустом месте: она перерастягивается, ускоряется и достигает экстремумов, где число активных участников уменьшается, а риск незаметно смещается на другую сторону рынка. Такие моменты обычно сначала проявляются на младших таймфреймах, где свинговые максимумы и минимумы начинают группироваться, а цена выходит за пределы недавней структуры. Индикатор свинговых экстремумов и откатов построен на следующем принципе: вместо следования за направлением он выявляет перерастяжение — моменты, когда цена слишком далеко и быстро отклоняется от непосредственной рыночной структуры.

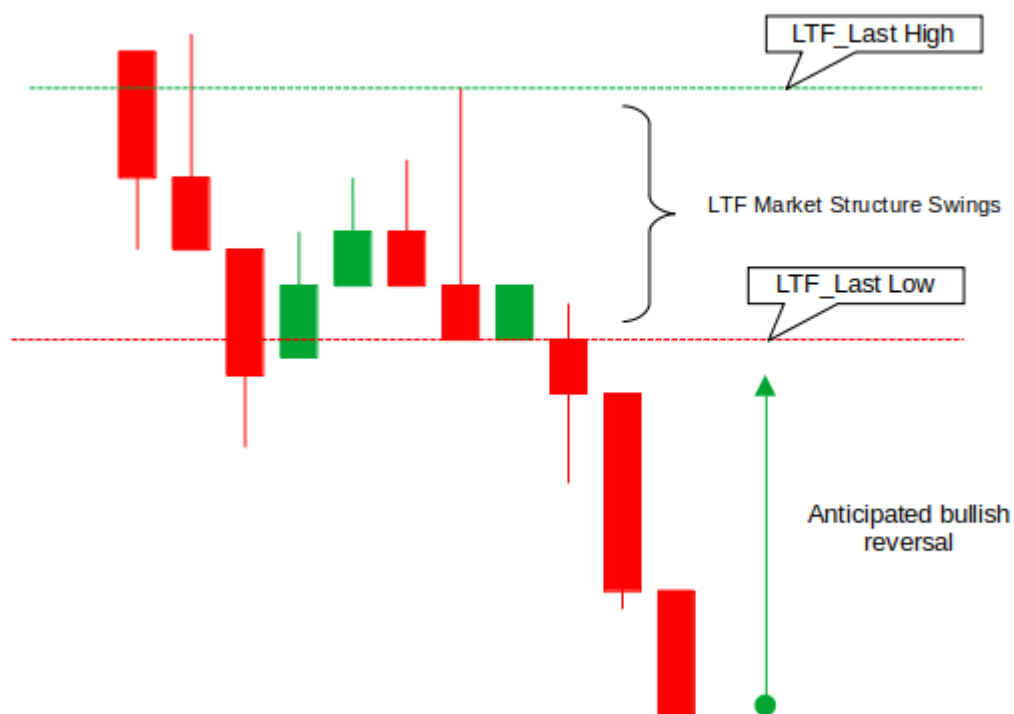
Анализируя свинговое поведение младшего таймфрейма, индикатор определяет, где цена находится выше или ниже недавних структурных границ, и отмечает условия, при которых откаты и возврат к среднему становятся статистически вероятнее. Вместо прогнозирования вершин или оснований индикатор реагирует на формирующиеся структурные признаки, преобразуя микроструктуру рынка в практические зоны для работы на разворот. В результате получается система, в которой развороты рассматриваются не как догадки, а как измеримая реакция на достижение ценой собственных экстремумов.

Обзор индикатора и принципы работы

Индикатор свинговых экстремумов и откатов: развороты цены обычно возникают после явного перерастяжения за пределы недавней структуры младшего таймфрейма. В обоих случаях индикатор сначала определяет последний свинговый максимум и свинговый минимум LTF, которые вместе задают краткосрочный структурный диапазон. Этот диапазон отражает "справедливое" поведение цены на младшем таймфрейме — область, в которой рынок ранее принимал данный ценовой уровень и где цена обычно совершала нормальные колебания.

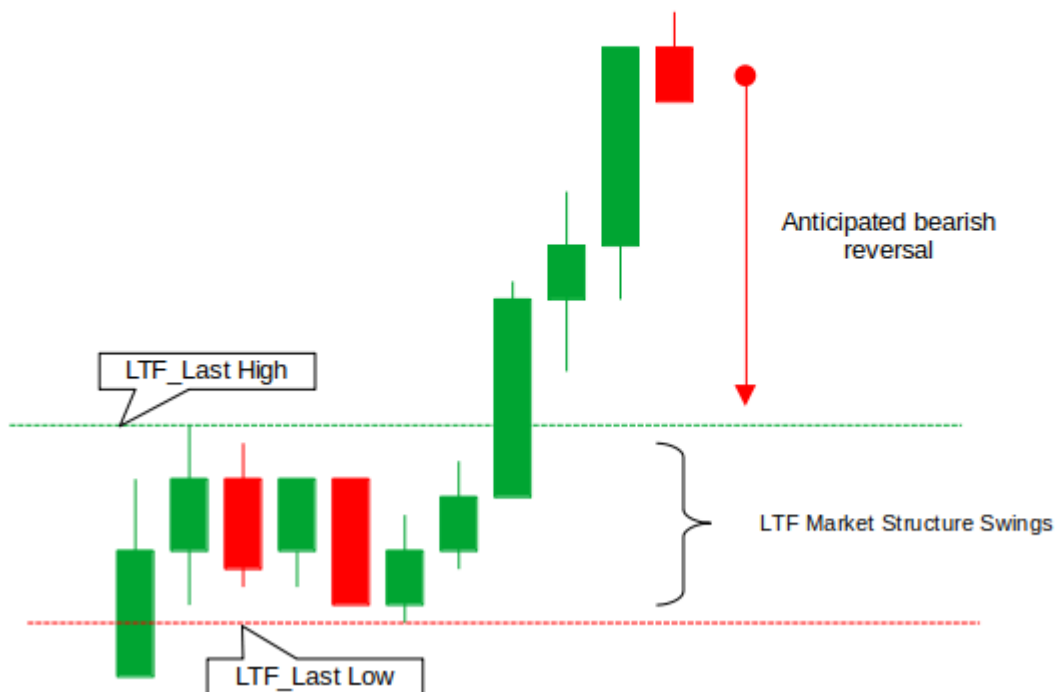
При экстремальном медвежьем движении цена резко опускается ниже последнего свингового минимума LTF, выходя далеко за пределы недавней структуры. Этот пробой вниз рассматривается не как сигнал продолжения, а как признак истощения. Когда цена находится значительно ниже последнего свингового минимума и более широкой зоны структуры LTF, индикатор отмечает условия статистического перерастяжения давления продавцов. Это повышает вероятность бычьего отката или возврата к среднему в направлении предыдущей структуры.

Extreme Bearish Price Move



И наоборот, экстремальное бычье движение демонстрирует противоположное поведение. Цена ускоряется выше последнего свингового максимума LTF, выходя за пределы предыдущей структуры и переходя в состояние перерастяжения. Вместо следования за пробоем индикатор интерпретирует это смещение как уязвимость — зону, в которой давление покупателей может ослабнуть, а медвежий откат становится более вероятным. В обоих случаях индикатор не пытается предсказывать точные вершины или основания. Он сосредоточен на выявлении структурного дисбаланса, позволяя рассматривать развороты как реакцию на достижение ценой собственных экстремумов, а не как произвольные контртрендовые сделки.

Extreme Bullish Price Move



Начало работы

```
//+-----+
//|                                     Swing Extremes.mq5 |
//|                                     Copyright 2025, MetaQuotes Ltd. |
//|                                     https://www.mql5.com/ru/users/johnhlomohang/ |
//+-----+
#property copyright "Copyright 2025, MetaQuotes Ltd."
#property link      "https://www.mql5.com/ru/users/johnhlomohang/"
#property version   "1.00"
#property indicator_chart_window
#property indicator_buffers 2
#property indicator_plots  2

#property indicator_label1  "Buy Signal"
#property indicator_type1   DRAW_ARROW
#property indicator_color1  clrLime
#property indicator_width1  2

#property indicator_label2  "Sell Signal"
#property indicator_type2   DRAW_ARROW
#property indicator_color2  clrRed
#property indicator_width2  2

//+-----+
//| Inputs |
//+-----+
input ENUM_TIMEFRAMES HTF = PERIOD_H1;
input ENUM_TIMEFRAMES LTF = PERIOD_M5;
```

```

input int          SwingBars = 3;
input int          ATR_Period   = 14;           // ATR calculation period
input double       ATR_Multiplier = 1.0;        // How extreme price must move beyond
input bool         Visualize = true;

//+-----+
//| Indicator Buffers |
//+-----+
double BuyBuffer[];
double SellBuffer[];

```

Для начала определим индикатор как инструмент окна графика с двумя буферами, предназначенными для визуального отображения торговых сигналов. Эти буферы настроены на отрисовку стрелок вверх и вниз, обозначающих условия покупки и продажи. Цветовая кодировка и размер стрелок делают сигналы разворота сразу заметными на графике. Затем задаются ключевые входные параметры — старший и младший таймфреймы, глубина обнаружения свингов и опциональный переключатель визуализации. Это позволяет индикатору адаптироваться к различным рыночным условиям, сохраняя гибкость логики на основе рыночной структуры. Параметры ATR_Period и ATR_Multiplier помогают выявлять экстремальные ценовые движения относительно структуры. Наконец, объявляются буферы покупки и продажи для хранения и отображения значений сигналов, формируемых расчетами свингов и рыночной структуры.

```

//+-----+
//| Swing Point Structure |
//+-----+
struct SwingPoint
{
    datetime time;
    double price;
    int type; // 1 = High, -1 = Low
    int barIndex;

    void Reset()
    {
        time = 0;
        price = 0;
        type = 0;
        barIndex = 0;
    }
};

//+-----+
//| Market Structure State |
//+-----+
enum STRUCTURE_STATE
{
    STRUCT_NEUTRAL,
    STRUCT_BULLISH,
    STRUCT_BEARISH
};

//+-----+
//| Globals |
//+-----+
SwingPoint htf_lastHigh, htf_prevHigh;
SwingPoint htf_lastLow, htf_prevLow;

```

```

SwingPoint ltf_lastHigh, ltf_prevHigh;
SwingPoint ltf_lastLow, ltf_prevLow;

STRUCTURE_STATE htfBias = STRUCT_NEUTRAL;
STRUCTURE_STATE ltfStructure = STRUCT_NEUTRAL;
int atrHandle;
datetime lastBuySwingTime = 0;
datetime lastSellSwingTime = 0;
double lastBuySwingPrice = 0;
double lastSellSwingPrice = 0;

```

В этом разделе рассматриваются основные структуры данных для аккуратного и повторно используемого представления рыночной структуры. Структура SwingPoint содержит всю необходимую индикатору информацию о свинговом максимуме или минимуме: время формирования, ценовой уровень, тип — максимум или минимум — и индекс бара формирования. Объединение этой информации в одной структуре и метод Reset() позволяют безопасно очищать и заново формировать данные свингов при инвалидации или пересчете структуры.

Направленность рынка задается перечислением STRUCTURE_STATE, классифицирующим поведение цены как бычье, медвежье или нейтральное на основе соотношения последовательных свинговых максимумов и минимумов. В рамках этой системы для HTF и LTF поддерживаются отдельные глобальные переменные для отслеживания свингов, позволяющие индикатору сравнивать более широкую рыночную направленность с локальной структурой. Дополнительные глобальные переменные хранят время и цену последнего свинга, использованного для покупки или продажи. Благодаря этому сигналы формируются только один раз для каждого структурного события и предотвращаются повторные срабатывания от одного свингового экстремума.

```

//+-----+
//| Indicator Init |
//+-----+
int OnInit()
{
    atrHandle = iATR(_Symbol, LTF, ATR_Period);

    SetIndexBuffer(0, BuyBuffer, INDICATOR_DATA);
    SetIndexBuffer(1, SellBuffer, INDICATOR_DATA);

    PlotIndexSetInteger(0, PLOT_ARROW, 233); // arrow up
    PlotIndexSetInteger(1, PLOT_ARROW, 234); // arrow down

    ArrayInitialize(BuyBuffer, EMPTY_VALUE);
    ArrayInitialize(SellBuffer, EMPTY_VALUE);

    return INIT_SUCCEEDED;
}

//+-----+
//| Indicator Calculation |
//+-----+
int OnCalculate(const int rates_total,
               const int prev_calculated,
               const datetime &time[],
               const double &open[],
               const double &high[],
               const double &low[],
               const double &close[],

```

```

        const long &tick_volume[],
        const long &volume[],
        const int &spread[])
{
    static datetime lastLtfBar = 0;
    datetime currentLtfBar = iTime(_Symbol, LTF, 0);

    if(currentLtfBar == lastLtfBar)
        return rates_total;

    lastLtfBar = currentLtfBar;

    UpdateHTFStructure();
    UpdateLTFStructure();
    CheckStructureBreak();

    int bar = rates_total - 1;

    BuyBuffer[bar] = EMPTY_VALUE;
    SellBuffer[bar] = EMPTY_VALUE;

    if(CheckBuyCondition())
        BuyBuffer[bar] = low[bar] - (10 * _Point);

    if(CheckSellCondition())
        SellBuffer[bar] = high[bar] + (10 * _Point);

    if(Visualize)
        UpdateVisualization();

    return rates_total;
}

```



В этом разделе инициализируется и реализуется основная рабочая логика индикатора. В функции OnInit инициализируется индикатор ATR. Затем буферы сигналов покупки и продажи привязываются к графическим построениям индикатора, настраиваются на отображение стрелок вверх и вниз и предварительно заполняются пустыми значениями для предотвращения нежелательных сигналов. Затем функция OnCalculate обрабатывает индикатор только при появлении нового бара младшего таймфрейма. Это обеспечивает эффективные обновления и предотвращает повторные пересчеты на каждом тике. На каждом новом баре LTF индикатор обновляет рыночную структуру старшего и младшего таймфреймов, проверяет пробой структуры и оценивает условия покупки и продажи. При обнаружении корректного сетапа для наглядности отрисовывается сигнальная стрелка немного выше или ниже текущего бара; при необходимости также обновляются все визуальные элементы структуры на графике.

```

//+-----+
//| Update Higher Timeframe Structure |
//+-----+
void UpdateHTFStructure()
{
    // Get HTF data
    MqlRates htfRates[];
    ArraySetAsSeries(htfRates, true);
    CopyRates(_Symbol, HTF, 0, 100, htfRates);

    if(ArraySize(htfRates) < SwingBars * 2 + 1) return;
}

```



```

// Detect swing points on HTF
for(int i = SwingBars; i < ArraySize(htfRates) - SwingBars; i++)
{
    bool isHigh = true;
    bool isLow = true;

    // Check if current bar is swing high
    for(int j = 1; j <= SwingBars; j++)
    {
        if(htfRates[i].high <= htfRates[i-j].high ||
           htfRates[i].high <= htfRates[i+j].high)
        {
            isHigh = false;
        }

        if(htfRates[i].low >= htfRates[i-j].low ||
           htfRates[i].low >= htfRates[i+j].low)
        {
            isLow = false;
        }
    }

    // Update swing highs
    if(isHigh)
    {
        htf_prevHigh = htf_lastHigh;
        htf_lastHigh.time = htfRates[i].time;
        htf_lastHigh.price = htfRates[i].high;
        htf_lastHigh.type = 1;
        htf_lastHigh.barIndex = i;
    }

    // Update swing lows
    if(isLow)
    {
        htf_prevLow = htf_lastLow;
        htf_lastLow.time = htfRates[i].time;
        htf_lastLow.price = htfRates[i].low;
        htf_lastLow.type = -1;
        htf_lastLow.barIndex = i;
    }
}

// Update HTF bias
htfBias = DetectStructure(htf_lastHigh, htf_prevHigh, htf_lastLow, htf_prevLow);
}

//+-----+
//| Update Lower Timeframe Structure |
//+-----+
void UpdateLTFStructure()
{
    // Get LTF data
    MqlRates ltfRates[];
    ArraySetAsSeries(ltfRates, true);
    CopyRates(_Symbol, LTF, 0, 100, ltfRates);
}

```



```

if(ArraySize(ltfRates) < SwingBars * 2 + 1) return;

// Detect swing points on LTF
for(int i = SwingBars; i < ArraySize(ltfRates) - SwingBars; i++)
{
    bool isHigh = true;
    bool isLow = true;

    // Check if current bar is swing high
    for(int j = 1; j <= SwingBars; j++)
    {
        if(ltfRates[i].high <= ltfRates[i-j].high ||
            ltfRates[i].high <= ltfRates[i+j].high)
        {
            isHigh = false;
        }

        if(ltfRates[i].low >= ltfRates[i-j].low ||
            ltfRates[i].low >= ltfRates[i+j].low)
        {
            isLow = false;
        }
    }

    // Update swing highs
    if(isHigh)
    {
        ltf_prevHigh = ltf_lastHigh;
        ltf_lastHigh.time = ltfRates[i].time;
        ltf_lastHigh.price = ltfRates[i].high;
        ltf_lastHigh.type = 1;
        ltf_lastHigh.barIndex = i;
    }

    // Update swing lows
    if(isLow)
    {
        ltf_prevLow = ltf_lastLow;
        ltf_lastLow.time = ltfRates[i].time;
        ltf_lastLow.price = ltfRates[i].low;
        ltf_lastLow.type = -1;
        ltf_lastLow.barIndex = i;
    }
}

// Update LTF structure
ltfStructure = DetectStructure(ltf_lastHigh, ltf_prevHigh, ltf_lastLow, ltf_prevLow)
}

```



Эти две функции отвечают за извлечение рыночной структуры из ценовых данных путем определения свинговых максимумов и минимумов на старшем и младшем таймфреймах. Каждая функция начинает с загрузки фиксированного окна исторических свечей и проверки достаточности данных для оценки свингов на основе заданной глубины SwingBars. Затем логика обнаружения свингов анализирует каждый подходящий бар и сравнивает его максимум и минимум с соседними свечами с обеих сторон, подтверждая свинговый максимум или минимум только при явном выделении цены относительно

окружающей структуры. При обнаружении нового свинга предыдущий сохраняется, а последний обновляется. Тем самым самым поддерживается непрерывно обновляемая история структуры, а не одна изолированная точка.

После обновления свинговых точек функции преобразуют необработанные ценовые данные в контекст направления рынка. На старшем таймфрейме оцениваются последний и предпоследний свинги для определения более широкой рыночной направленности, тогда как структура младшего таймфрейма классифицируется отдельно для отражения краткосрочного поведения. Такое разделение позволяет индикатору сопоставлять развороты микроуровня со структурой макроуровня. Одна и та же логика свингов на разных таймфреймах формирует целостное представление о тренде, консолидации или перерастяжении без опоры на запаздывающие индикаторы.

```
//+-----+
//| Detect Market Structure |
//+-----+
STRUCTURE_STATE DetectStructure(SwingPoint &lastHigh, SwingPoint &prevHigh,
                               SwingPoint &lastLow, SwingPoint &prevLow)
{
    // Need at least two highs and two lows
    if(lastHigh.price == 0 || prevHigh.price == 0 ||
        lastLow.price == 0 || prevLow.price == 0)
        return STRUCT_NEUTRAL;

    bool isHigherHigh = lastHigh.price > prevHigh.price;
    bool isHigherLow = lastLow.price > prevLow.price;
    bool isLowerLow = lastLow.price < prevLow.price;
    bool isLowerHigh = lastHigh.price < prevHigh.price;

    if(isHigherHigh && isHigherLow)
        return STRUCT_BULLISH;

    if(isLowerLow && isLowerHigh)
        return STRUCT_BEARISH;

    return STRUCT_NEUTRAL;
}

//+-----+
//| New Sell Condition |
//+-----+
bool CheckSellCondition()
{
    // Only allow sells if HTF bias is BEARISH
    if(htfBias != STRUCT_BEARISH)
        return false;

    // Need valid LTF swing points
    if(ltf_lastHigh.price == 0 || ltf_lastLow.price == 0)
        return false;

    double currentPrice = SymbolInfoDouble(_Symbol, SYMBOL_BID);

    double atr = GetCurrentATR();
    if(atr == 0) return false;

    bool priceAboveHigh = currentPrice > (ltf_lastHigh.price + atr * ATR_Multiplier);
    bool priceAboveLow = currentPrice > ltf_lastLow.price;
```



```

    bool isNewSwing = (ltf_lastHigh.time != lastSellSwingTime) ||
                      (ltf_lastHigh.price != lastSellSwingPrice);

    return (priceAboveHigh && priceAboveLow && isNewSwing);
}

//+-----+
//| New Buy Condition |
//+-----+
bool CheckBuyCondition()
{
    // Only allow buys if HTF bias is BULLISH
    if(htfBias != STRUCT_BULLISH)
        return false;

    // Need valid LTF swing point
    if(ltf_lastLow.price == 0)
        return false;

    double currentPrice = SymbolInfoDouble(_Symbol, SYMBOL_ASK);

    double atr = GetCurrentATR();
    if(atr == 0) return false;

    bool priceBelowLow = currentPrice < (ltf_lastLow.price - atr * ATR_Multiplier);

    bool isNewSwing = (ltf_lastLow.time != lastBuySwingTime) ||
                      (ltf_lastLow.price != lastBuySwingPrice);

    return (priceBelowLow && isNewSwing);
}

```



В этом разделе описывается, как необработанные данные свингов преобразуются в значимый рыночный контекст. Функция DetectStructure оценивает соотношение между последовательными свинговыми максимумами и минимумами, чтобы классифицировать поведение цены как бычье, медвежье или нейтральное. Бычья структура подтверждается только при наличии более высокого максимума и более высокого минимума, тогда как для медвежьей структуры требуются более низкий минимум и более низкий максимум. Если эти условия не выполняются, рынок считается нейтральным. Это предотвращает предположения о направлении при неясной или неполной рыночной структуре.

Функции условий покупки и продажи опираются на эту структурную основу, одновременно учитывая согласованность с направленностью HTF и порог перерастяжения на основе ATR. Условие продажи рассматривается только при медвежьей направленности старшего таймфрейма и выходе цены выше последнего свингового максимума LTF более чем на заданный множитель ATR. При этом цена остается выше соответствующей свинговой структуры, что указывает на экстремум с поправкой на волатильность, а не на незначительный пробой. Аналогичным образом, условие покупки считается подтвержденным только при бычьей направленности HTF и снижении цены ниже последнего свингового минимума LTF на расстояние с поправкой на ATR. Это указывает на значимое перерастяжение вниз. В обоих случаях логика гарантирует, что каждый свинговый экстремум используется для сделки только один раз, сверяя ранее сохраненные время и цену свинга. Благодаря этому индикатор фокусируется на структурных неповторяющихся возможностях разворота, а не реагирует на незначительные колебания цены.

При получении сигнала сначала следует дождаться его согласованности с направленностью HTF, отображаемой на графике. Сделки валидны только в направлении этой направленности: бычья

допускает только покупки, а медвежья — только продажи. Торговое условие считается активным, когда цена выходит за пределы соответствующего свингового экстремума LTF, отображаемого индикатором: ниже последнего свингового минимума LTF для покупок или выше последнего свингового максимума и минимума LTF для продаж. Метка условия должна подтверждать активный сетап. Для управления рисками стоп-лоссы можно размещать за соответствующим свинговым экстремумом LTF: выше последнего свингового максимума LTF для продаж и ниже последнего свингового минимума LTF для покупок. Эти уровни являются точками структурной инвалидации в логике индикатора. Для фиксации прибыли по позициям на продажу можно ориентироваться на последний свинговый минимум LTF или середину диапазона между свингами. Для позиций на покупку ориентиром служит последний свинговый максимум LTF или срединный уровень диапазона — в зависимости от предпочтений по риску.

```
//+-----+
//| Check Structure Break                               |
//+-----+
void CheckStructureBreak()
{
    MqlRates current[];
    ArraySetAsSeries(current, true);
    CopyRates(_Symbol, LTF, 0, 1, current);

    if(ArraySize(current) < 1) return;

    // Bullish structure break
    if(ltfStructure == STRUCT_BULLISH && current[0].close < ltf_lastLow.price)
    {
        ResetLTFStructure();
    }

    // Bearish structure break
    if(ltfStructure == STRUCT_BEARISH && current[0].close > ltf_lastHigh.price)
    {
        ResetLTFStructure();
    }
}

//+-----+
//| Reset LTF Structure                               |
//+-----+
void ResetLTFStructure()
{
    ltf_lastHigh.Reset();
    ltf_prevHigh.Reset();
    ltf_lastLow.Reset();
    ltf_prevLow.Reset();
    ltfStructure = STRUCT_NEUTRAL;

    // Also reset trade tracking
    lastBuySwingTime = 0;
    lastSellSwingTime = 0;
    lastBuySwingPrice = 0;
    lastSellSwingPrice = 0;
}

//+-----+
//| Update Visualization                               |
//+-----+
```



```

void UpdateVisualization()
{
    ObjectsDeleteAll(0, "MS_");

    // Draw HTF swing points
    if(htf_lastHigh.price > 0)
        DrawSwingPoint(htf_lastHigh, "HTF_High", clrRed, HTF);

    if(htf_prevHigh.price > 0)
        DrawSwingPoint(htf_prevHigh, "HTF_PrevHigh", clrRed, HTF);

    if(htf_lastLow.price > 0)
        DrawSwingPoint(htf_lastLow, "HTF_Low", clrBlue, HTF);

    if(htf_prevLow.price > 0)
        DrawSwingPoint(htf_prevLow, "HTF_PrevLow", clrBlue, HTF);

    // Draw LTF swing points
    if(ltf_lastHigh.price > 0)
        DrawSwingPoint(ltf_lastHigh, "LTF_High", clrOrange, LTF);

    if(ltf_prevHigh.price > 0)
        DrawSwingPoint(ltf_prevHigh, "LTF_PrevHigh", clrOrange, LTF);

    if(ltf_lastLow.price > 0)
        DrawSwingPoint(ltf_lastLow, "LTF_Low", clrGreen, LTF);

    if(ltf_prevLow.price > 0)
        DrawSwingPoint(ltf_prevLow, "LTF_PrevLow", clrGreen, LTF);

    // Draw current price lines for reference
    double currentBid = SymbolInfoDouble(_Symbol, SYMBOL_BID);
    double currentAsk = SymbolInfoDouble(_Symbol, SYMBOL_ASK);

    CreateHorizontalLine("Current_Bid", currentBid, clrYellow, STYLE_DASH);
    CreateHorizontalLine("Current_Ask", currentAsk, clrYellow, STYLE_DASH);

    // Draw trade condition zones
    if(ltf_lastHigh.price > 0)
        CreateHorizontalLine("Sell_Zone_Min", ltf_lastHigh.price, clrRed, STYLE_SOLID);

    if(ltf_lastLow.price > 0)
    {
        CreateHorizontalLine("Buy_Zone_Max", ltf_lastLow.price, clrBlue, STYLE_SOLID);
        CreateHorizontalLine("Sell_TP_Level", ltf_lastLow.price, clrGreen, STYLE_DOT);
    }

    // Draw structure labels
    DrawStructureLabels();
}

```



В этом разделе выполняется проверка структуры и обеспечивается соответствие индикатора текущему поведению рынка. Функция CheckStructureBreak постоянно отслеживает последнюю свечу младшего таймфрейма и сравнивает цену ее закрытия с последними свинговыми уровнями. Если бычья структура invalidируется закрытием цены ниже последнего свингового минимума или медвежья структура нарушается закрытием цены выше последнего свингового максимума, существующая структура

считается несостоятельной. В ответ функция ResetLTFStructure сбрасывает и обновляет сигналы, не позволяя устаревшим взаимосвязям свингов влиять на будущие сигналы и обеспечивая использование только актуальных и не нарушенных структур.

После подтверждения или перестроения структуры функция UpdateVisualization преобразует внутреннюю логику в понятный графический контекст в реальном времени. Она перерисовывает свинговые точки старшего и младшего таймфреймов, ориентиры текущей цены и ключевые структурные уровни, определяющие зоны покупки и продажи. Визуально разделяя свинги HTF и LTF, а также накладывая уровни и метки для конкретных условий, индикатор позволяет сразу видеть не только место появления сигнала, но и его причину. Каждый сетап при этом опирается на наблюдаемую рыночную структуру, а не на абстрактные сигналы.

```
//+-----+
//| Draw Swing Point |
//+-----+
void DrawSwingPoint(SwingPoint &sp, string name, color clr, ENUM_TIMEFRAMES tf)
{
    ObjectCreate(0, "MS_" + name, OBJ_ARROW, 0, sp.time, sp.price);
    ObjectSetInteger(0, "MS_" + name, OBJPROP_ARROWCODE, sp.type == 1 ? 218 : 217);
    ObjectSetInteger(0, "MS_" + name, OBJPROP_COLOR, clr);
    ObjectSetInteger(0, "MS_" + name, OBJPROP_WIDTH, 3);
    ObjectSetString(0, "MS_" + name, OBJPROP_TOOLTIP,
        StringFormat("%s: %.5f", sp.type == 1 ? "High" : "Low", sp.price));
}

//+-----+
//| Draw Structure Labels |
//+-----+
void DrawStructureLabels()
{
    string biasText = "HTF Bias: ";
    switch(htfBias)
    {
        case STRUCT_BULLISH: biasText += "BULLISH"; break;
        case STRUCT_BEARISH: biasText += "BEARISH"; break;
        default: biasText += "NEUTRAL"; break;
    }

    string ltfText = "LTF Structure: ";
    switch(ltfStructure)
    {
        case STRUCT_BULLISH: ltfText += "HH+HL"; break;
        case STRUCT_BEARISH: ltfText += "LL+LH"; break;
        default: ltfText += "NEUTRAL"; break;
    }

    // Current trade conditions
    string conditionText = "";
    if(CheckBuyCondition())
        conditionText = "Potential BUY Condition: ACTIVE (Price < LTF Low)";
    else if(CheckSellCondition())
        conditionText = "Potential SELL Condition: ACTIVE (Price > LTF High & Low)";
    else
        conditionText = "No Anticipated Trade Condition Met";

    // Create label objects
    CreateLabel("MS_BiasLabel", biasText, 10, 20, clrDarkOrange);
```

```

CreateLabel("MS_LTFLabel", ltfText, 10, 40, clrDarkOrange);
CreateLabel("MS_ConditionLabel", conditionText, 10, 60,
            (CheckBuyCondition() || CheckSellCondition()) ? clrLime : clrGray);

// Display LTF swing prices
if(ltf_lastHigh.price > 0)
    CreateLabel("MS_LTFHighLabel", StringFormat("LTF High: %.5f", ltf_lastHigh.price)

if(ltf_lastLow.price > 0)
    CreateLabel("MS_LTFLowLabel", StringFormat("LTF Low: %.5f", ltf_lastLow.price), 1
}

//+-----+
//| Create Text Label |
//+-----+
void CreateLabel(string name, string text, int x, int y, color clr)
{
    ObjectCreate(0, "MS_" + name, OBJ_LABEL, 0, 0, 0);
    ObjectSetString(0, "MS_" + name, OBJPROP_TEXT, text);
    ObjectSetInteger(0, "MS_" + name, OBJPROP_XDISTANCE, x);
    ObjectSetInteger(0, "MS_" + name, OBJPROP_YDISTANCE, y);
    ObjectSetInteger(0, "MS_" + name, OBJPROP_COLOR, clr);
    ObjectSetInteger(0, "MS_" + name, OBJPROP_FONTSIZE, 10);
}

//+-----+
//| Create Horizontal Line |
//+-----+
void CreateHorizontalLine(string name, double price, color clr, ENUM_LINE_STYLE style)
{
    ObjectCreate(0, "MS_" + name, OBJ_HLINE, 0, 0, price);
    ObjectSetInteger(0, "MS_" + name, OBJPROP_COLOR, clr);
    ObjectSetInteger(0, "MS_" + name, OBJPROP_STYLE, style);
    ObjectSetInteger(0, "MS_" + name, OBJPROP_WIDTH, 1);
}

//+-----+
//| Get Current ATR |
//+-----+
double GetCurrentATR()
{
    double atr[];
    ArraySetAsSeries(atr, true);

    if(CopyBuffer(atrHandle, 0, 0, 1, atr) <= 0)
        return 0;

    return atr[0];
}

```

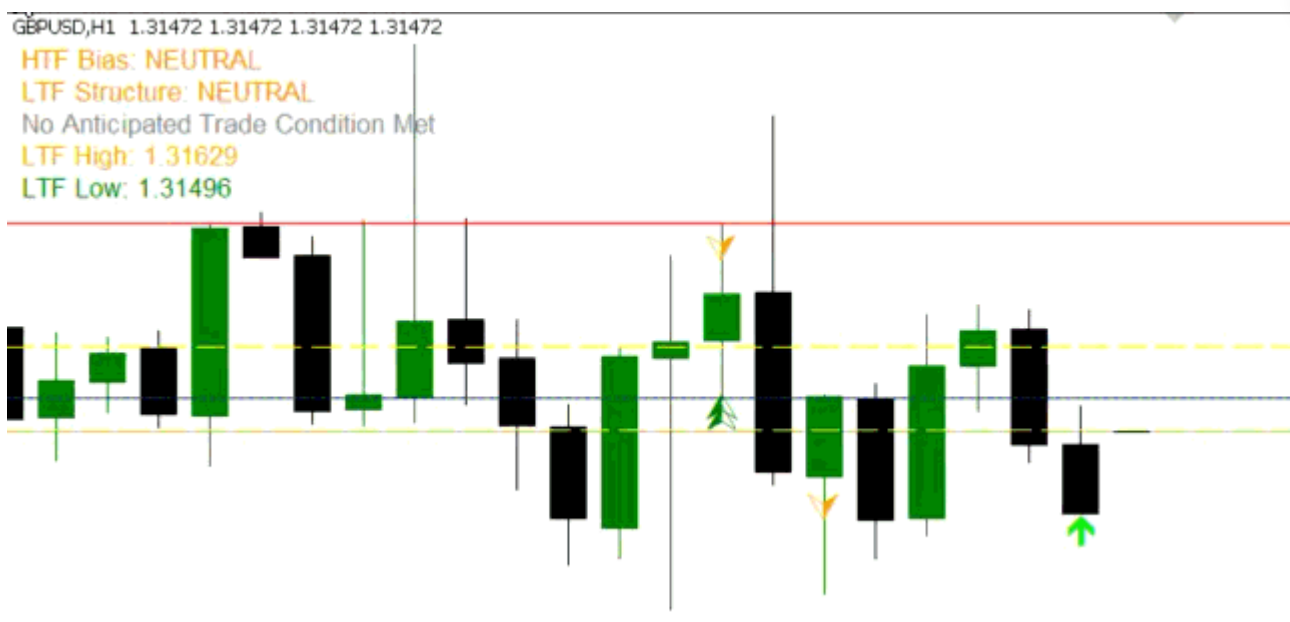


Этот последний раздел посвящен преобразованию внутренних данных о свингах и структуре в визуальное представление, которое можно сразу интерпретировать на графике. Функция DrawSwingPoint создает объекты-стрелки в каждой обнаруженной свинговой точке. Направление стрелки, цвет и всплывающая подсказка показывают, является ли точка вершиной или впадиной. Это позволяет легко увидеть точные моменты формирования значимого свинга, сокращая разрыв между необработанными числовыми данными и практически применимой информацией. Каждая свинговая точка четко

обозначена, что помогает понимать ценовые экстремумы и время их формирования без необходимости вручную анализировать свечные паттерны.

В дополнение к свинговым точкам функция DrawStructureLabels и вспомогательные функции CreateLabel и CreateHorizontalLine предоставляют динамическую сводку состояния рынка и потенциальных торговых условий. Отображаются направленности HTF и LTF, показывающие, является ли более широкий тренд бычьим, медвежьим или нейтральным, а также вошла ли текущая цена в потенциальную зону покупки или продажи. Горизонтальные линии обозначают критически важные свинговые максимумы и минимумы, а метки с цветовой кодировкой визуальнo выделяют зоны и условия. Затем функция GetCurrentATR получает и возвращает последнее значение ATR из буфера индикатора ATR или возвращает 0, если доступ к данным невозможен. В совокупности эти элементы формируют интерактивную карту рыночной структуры в реальном времени, помогающую выявлять возможные развороты и откаты.

Демонстрация индикатора



Заключение

Подводя итог, мы разработали индикатор свинговых экстремумов и откатов, объединяющий выявление свингов на нескольких таймфреймах с анализом рынка младшего таймфрейма в реальном времени. Индикатор выявляет ключевые свинговые максимумы и минимумы на старших и младших таймфреймах, определяет текущую рыночную структуру как бычью, медвежью или нейтральную и отмечает потенциальные зоны перерастяжения, в которых цена может развернуться. Визуальные ориентиры — стрелки для свинговых точек, горизонтальные линии для ключевых уровней и динамические метки направленности HTF, структуры LTF и активных торговых условий — позволяют легко отслеживать внутренний ритм рынка. Благодаря проверке структуры, контролю перерастяжения и сбросу свингов индикатор обеспечивает своевременность и релевантность сигналов, а также предотвращает их повторение на одних и тех же свинговых экстремумах.

Этот индикатор может повысить эффективность торговли, преобразуя необработанное ценовое движение в понятную и практически применимую карту потенциальных разворотов и откатов. Трейдеры могут заранее оценивать реакцию цены на свинговых экстремумах, быстро определять, благоприятствуют ли условия бычьим или медвежьим сетапам, и принимать обоснованные решения без ручного отслеживания свингов на нескольких таймфреймах. Визуализируя рыночную структуру и торговые зоны в реальном времени, индикатор уменьшает элемент догадки, упрощает анализ и

позволяет выполнять более уверенные и точные входы и выходы, связывая анализ рынка с практическим исполнением сделок. После прочтения статьи вы сможете как минимум определять экстремум LTF, фильтровать его с учетом направленности HTF и применять структурированные правила входа, установки стоп-лосса и выхода.

Следующая статья >>

Hlomohang John Borocho

Founder and CEO в Dynamic Financial Labs

[Лесото](#)

[15103](#)



Quantitative intraday trading strategies
· MQL5 Expert Advisor development ·
Python + AI automation for MetaTrader 5.
Built by a developer who actually trades.

Другие статьи автора

- Теория графов: Алгоритм Форда—Фалкерсона для оценки ликвидности и пропускной способности рынка
- Интеграция MQL5 с пакетами обработки данных (Часть 9): Адаптивная оценка волатильности на основе энтропии
- Теория графов: Применение алгоритма эвристического поиска A* в торговле
- Разработка динамического мультивалютного советника (Часть 8): Ротация капитала в зависимости от времени суток
- Интеграция MQL5 с пакетами обработки данных (часть 8): Использование графовых нейронных сетей для выявления зон ликвидности
- Теория графов: Применение поиска в глубину (DFS) в торговых системах
- Разработка динамического мультивалютного советника (Часть 7): Карта межпарных корреляций для фильтрации сделок в реальном времени

[Follow us](#) on socials for top
articles and CodeBase updates